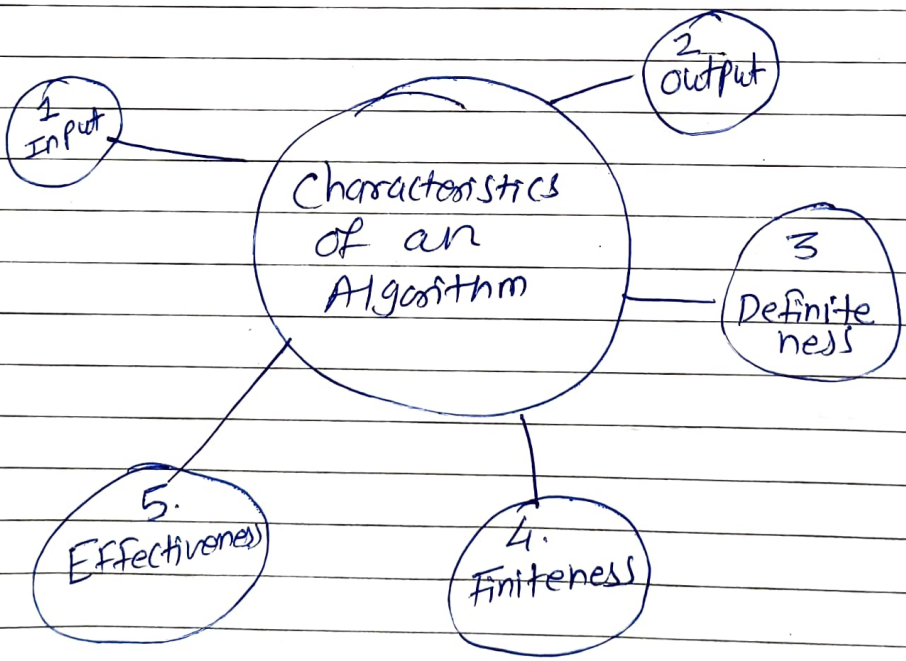
DAA BY
Unit I B.A. Khirsagar

Page No.
Date

* Role of Algorithm in computing —

- The algorithm is a set of rules which are used to solve real life problems.
- An algorithm is a finite set of unambiguous steps needed to be followed in certain order to accomplish a specific task.
- In the case of computational algorithms, these steps refer to the instructions that contains fundamental operations like +, −, *, /, % etc.
- Algorithm provide a precise description of the procedure to be followed in certain order to solve a well-defined computational problem.

* Characteristics of an Algorithm



1. Input — An algorithm has zero or more inputs. Each instruction which contains a fundamental operator must accept zero or more inputs.

SPPU ENGINEERING UPDATES

Study material provided by : Vishwajeet Londhe

Join community by clicking below links

@SPPU_ENGINEERING_UPDATE

Insta Page

https://www.instagram.com/sppu_engineering_update

@SPPU_TE_BE_COMP

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

WhatsApp Channel

https://whatsapp.com/channel/0029VaAhRMdAzNbmPG0jyq2x

2.) Output – An algorithm produces at least one output.
- Every instruction which contains a fundamental operator must accept zero or more inputs.

3) Definiteness - All instructions in an algorithm must be unambiguous, precise and easy to interpret.
By refering any of the instruction in an algo one can clearly understand what is to be done.

4) Finiteness – An algorithm must terminate after a finite number of steps in all test cases.
Every instruction which contains a fundamental operator must be terminated within a finite amount of time.

5) Effectiveness – An algorithm must be developed by using very basic, simple and feasible operations so that one can trace it out by using just paper & pencil.

Page No.
Date

* Evolution of Algorithm

- First ever algorithm was proposed by unknown Indian mathematician who used the concept of zero and decimal position of numbers.
- This algorithm provided a base for many basic arithmatic operations such as square root, cube roots.
- During 1940's & 1950's research was oriented towards building efficient computer systems, so that they can be used in scientific, commercial and engineering problems.
- Structural programming came into existance after Alan Turing introduced the idea of Effective procedure in 1936.
- Application of computers in the diverse field led the search for more efficient algorithms & study of tractable and non-tractable problems.
- Donald Knuth - father of algorithms - introduced the term algorithm analysis in his famous book "The Art of Computer Programming."
- Complexity theory - Algorithms are being analyzed on various bounds. The focus of researchers was on minimizing the lower bound running time of algorithms.
- Later correctness of algorithm come in picture.
- With the development of algorithms, limits of input & expected o/p of the algorithm is also given importance.

- Study & development of efficient algorithm led to another aspect called time - space tradeoff.
- Approximation algorithm came into picture to provide an approximate soln to non-tractable problems in reasonably acceptable time.
- Randomized algorithms have given a new direction for solving many time-consuming problems.
- Quick sort is sufficient to explain the importance of randomness.

Iterative algorithm design issues

→ The iteration may be explicit through a looping construct or it can be due to a recursive algorithm.

→ Even a small amount of excess time spent within the loop can lead to a major loss in the efficiency of amount of excess time spent within the loop can lead to a major loss in the efficiency of the algorithm.

→ The situation can even be worse if there are loops within loops or nestested algorithm.

→ Design issues of the iterative algorithms:

1. Iterations using the loop control structure
2. Improving the efficiency of the algorithms
3. Estimation of time and space requirements
4. Expressing the complexities using order notations
5. Applying different algorithmic strategies

1) Iterations using the Loop control structure:

For any loop we specify

1. Initial condition – that needs to be true before the loop begins execution
2. The Invariant relation that must hold before, during, and after each iteration of the loop
3. The condition under which the loop must terminate.

eg.

```
i ← 0
Sum ← 0
while (i ≤ n)          // n is known value
    i ← i + 1
    Sum ← Sum + a[i]
endWhile.
```

if we put $n = 5$, we know loop will execute 5 times. Means it is known in advance the no. of times the loop is to be iterated.

Another possibility is that loop terminates when some condition becomes false.

eg.

```
while (x > 0) and (x < 10) do
  ...
  ...
endWhile.
```

For loops of this type we cannot directly predict in advance the no. of times the loop will iterate before it terminates.

2) Improving efficiency of algo :-

Efficiency in terms of Time & space

Techniques to improve algo efficiency –

(1) Removing redundant Computations outside loop

- The most common mistake when using loops is to repeatedly re-calculate the part of an expression that remains constant throughout the execution phase of the entire loop

eg.

```
x = 0;
for i = 1 to N do
{   x = x + 1;
    Y = (a*a*a + b + c) * x * z;
    Printf (Y);
}
```

How we can remove unnecessary multiplications & computations.

```
q1 = q * q * a + c;
x = 0;
for i = 1 to N do
{
    x = x + 1;
    y = q1 * x * x;
    print (y);
}
```

(2) Referencing of Array Elements.

→ Generally, arrays are processed by iterative constructs.

→ If care is not taken while programming, redundant computations can grow into array processing.

eg

First version

```
j = 0
for (i=1; i<n; i++)
{
if (a[i] > a[j])
    {
    j = i;
    }
}
max = a[j].
```

Second version

```
max = a[0];
for (i=1; i<n; i++)
{
if (a[i] > max)
    {
    max = a[i];
    }
}
```

→ In Second version - only one array referencing is used.

→ References to array elements requires address arithmatic to arrive at the correct element and this requires time.

(3) Inefficiency due to late termination
← Inefficiency can be carried out when more tests are carried out than are required to solve the problem at hand.

eg. → The inefficiency into bubble sort algorithm if care is not taken while implementing it. This can happen if the inner loop that drives the exchange mechanism always goes through the full length of the array.

eg.

```
for i = 1 to n-1
    for j = 1 to n-1
if (a[j] > a[j+1]) then
    exchange a[j] with a[j+1].
```

With this sorting mechanism, after ith iteration the last i values in the array will be in sorted order. Thus for any given i, the inner loop should not proceed beyond n-i.
Hence the loop structure should be -

```
for i = 1 to n-1
    for j = 1 to n-i
        if (a[j] > a[j+1]) then
            exchange a[j] with a[j+1].
```

(4) Early detection of desired o/p conditions.

Next page ⇒

- For a certain i/p instances of a problem, the algorithm can reach the expected output condition before its regular condition of termination is reached.
- Identifying such conditions, the algorithm can have an early exit from a loop.
- It Saves a computational time.

eg.

Find atleast one even no. from given array A[0:n-1]

```
for (i=0; i<n; i++)
{ if ((A[i]%2)==0)
    write (A[i])
}
```

Not efficient code

```
For (i=0; i<n; i++)
{ if ((A[i]%2)==0)
  { Write (A[i]);
  } break;
}
```

efficient code

* Algorithm as a technology:-

- Computing time & memory space are limited resources, so we should use them sensibly.
- It is achieved by the usage of efficient algorithms that need less space and time.
- Different algo take diff time to solve the same problem.
- Consider a faster computer A (executes 10^{10} instruction/second).
- Slower computer B (executes 10^7 instruction/second)

⇒ Let insertion sorts running time is $c_1 \cdot n^2$ & merge sort's running time as $c_2 \cdot n \cdot \log n$.

- Suppose c_1 is 2 & c_2 is 50.
- Faster (A) computer running Insersion sort
- Slower (B) computer running merge sort
- A is 1000 times faster than B
- To sort 10 million numbers A. takes

$$= \frac{2 \cdot (10^7)^2 \text{ instructions}}{10^{10} \text{ instructions/second}}$$

$$= 20{,}000 \text{ Seconds.} \quad (\text{more than } 5.5 \text{ hours})$$

- B takes

$$= \frac{50 \cdot 10^7 \cdot \log 10^7 \text{ instructions}}{10^7 \text{ instruction/second}}$$

$\therefore$ 1163 Seconds
(less than 20 ~~second~~ Minutes)

By using an algo whose running time grows more slowly, even with a poor compiler, computer B runs more than 17 times faster than computer A.

- Eg above shows that we should consider algorithm, like computer h/w, as a technology.
- System performance depends on choosing efficient algorithms as much as on choosing

Page No.
Date

Fast hardware.

Algorithm are important as -

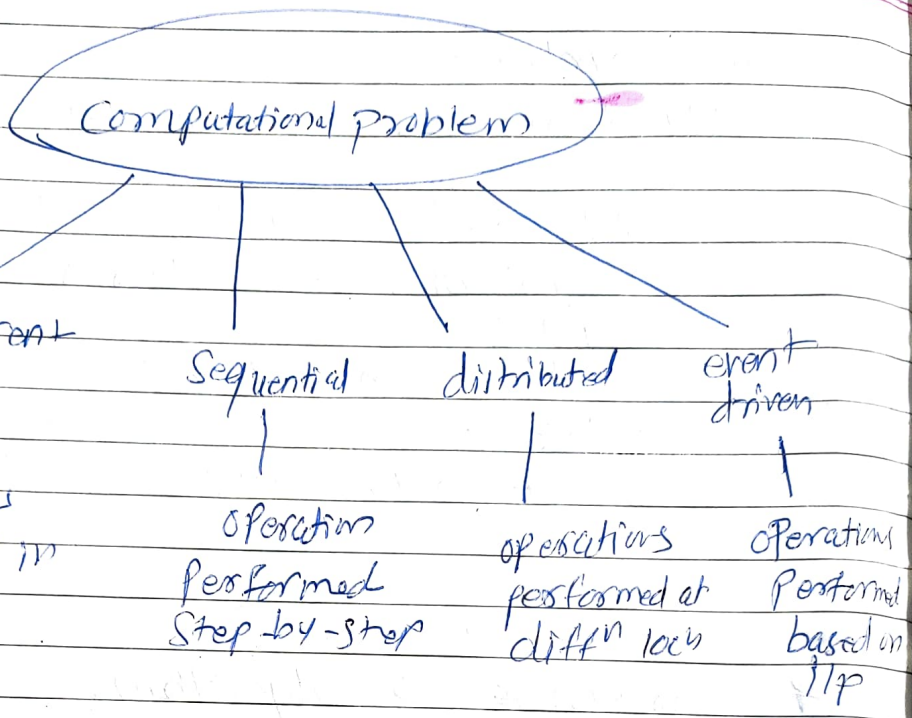
- Advanced Computer Architecture & fabrication technologies
- GUI
- object oriented system
- integrated web Technologies
- fast networking (wired & wireless)

* Correct & incorrect algorithms

- Algorithm is correct if, for every i/p instance, it ends with the correct o/p.
- Incorrect algo might not end at all on some i/p instances, or it might end with an answer other than the desired one.

* classification of problems

- Problems based on algorithmic soln
  - ↓
  - Step by Step instruction & series of actions
  - ↓
  - Sorting algo
- problems based on Heuristic soln.
  - ↓
  - trial & error
  - ↓
  - Which stock to buy?



Problem Solving Strategies

1) Divide & conquer – Solving a large problem by breaking the problem into smaller sub-problems, solving the sub-problems & combining the soln.
eg - Binary Search, Merge sort

2) Greedy – Solving problem by selecting the best option available from given options
eg - Fractional Knapsack Problem
Job sequencing

3) Dynamic – Solving problem by breaking them down into a collection of simpler subproblems, solving each of those subproblems just once, and storing their soln for future use.
eg - 0/1 Knapsack, OBST

4) Branch & bound (~~Red Green~~) – Total set of soln can be partitioned into smaller subsets of soln
eg - TSP

5) Backtracking – It uses recursive calling to find soln by building a soln step by step increasing values with time.
eg → N-queens problem.

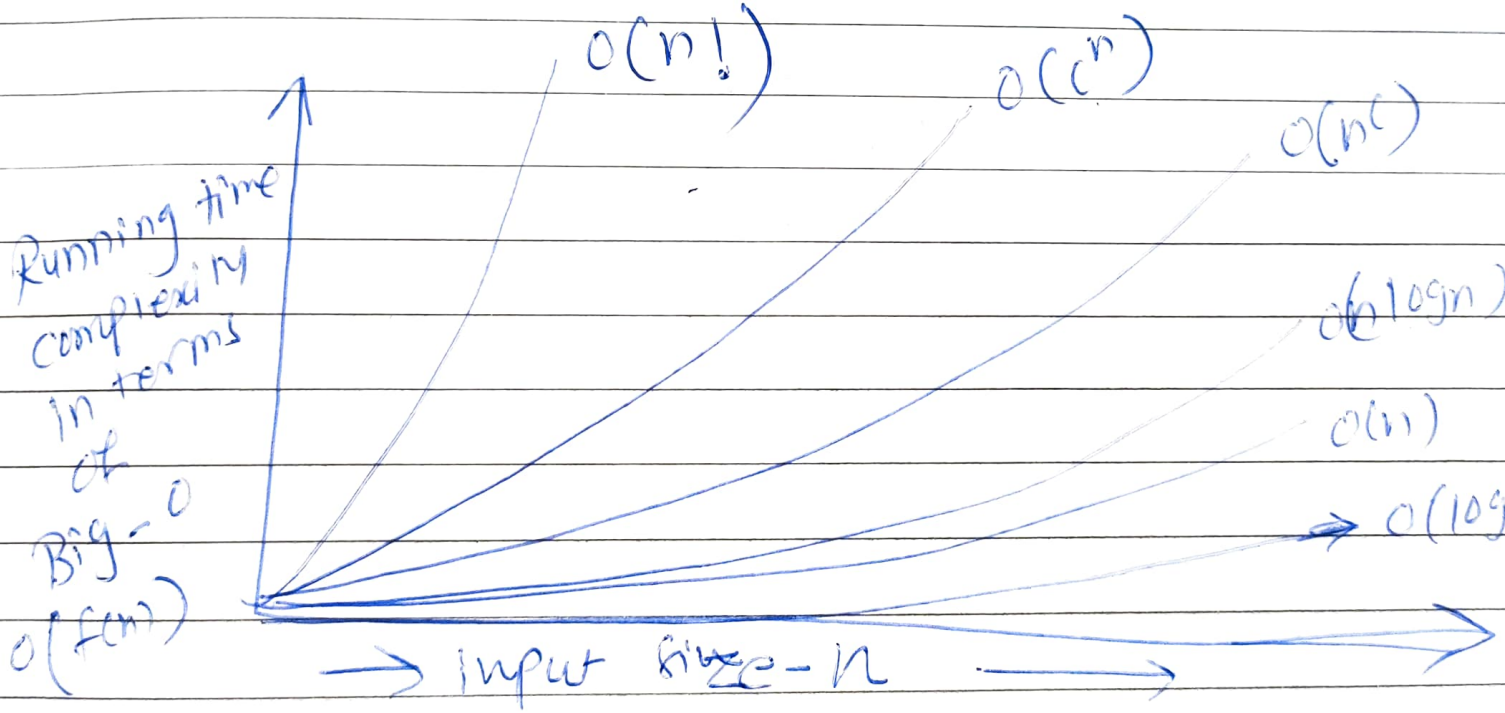
- Analysis of algorithms:
  - We can compare one algo with other and choose the best. This is called as analysis of algo.
  - analysis involves measuring the performance of an algo.
  - performance is measured in terms of
    1. programmers time complexity —
    2. Time complexity —
       The amount of time taken by algo to perform the intended task.
    3. Space complexity: —
       - The amount of memory need to perform the task.
       - Space complexity is computed by considering the data and their sizes.

- complexity of algo: —
  - algo are measured in terms of time and space complexity.

1. Time complexity: —
   - The max. time required for the execution of prog. as a function of the i/p size.

Classification of time complexities

- Constant — $O(1)$
- Linear — $O(n)$
- Logarithmic — $O(\log n)$
- Quadratic — $O(n^2)$
- Exponential — $O(2^n)$



slow

- Time complexity ~~function~~ ~~T(P)~~ is the time taken by program P and is the sum of the compile & execution time.
- We can determine the no of steps needed by prog to solve particular problem instance in one of two ways
  1) By introducing one variable as global & initializing it as zero and increment it by each statement in prog.
  2) compute NO of times each statement will be executed manually. NO of times the statement is executed is its frequency count Sum freq count of all statement This sum is no of steps needed to solve given problem.
- An algo. can be characterized by a timing fun $T(N)$. $T(N)$ is a measure

of how much time is required to execute an algo given N values.

(2) Space complexity
- Space complexity specifies the amount of computer memory required during the program execution as a function of I/P size.
- space complexity measure at two different times:
  (a) compile time
  (b) Run time.

(a) compile time space complexity :-
- Storage required of a program at compile time.
- it is determined by summing up the storage size of each variable using declaration statements.
- This includes memory requirement before execution starts.

(b) Runtime space complexity -
- If program is recursive or uses dynamic variables or dynamic data structure, then there is need to determine Runtime space complexity.
- The memory requirement is summation of the program space, data space and stack space.

Page No.
Date

Find Time & Space complexity example

Frequency count Example

(1)

	Frequency	total
Fun Sum 1	-	
{	-	
Sum = 0	1	1
for (i = 1; i <= n i++)	i = 1 ⇒ 1 i <= 1 ⇒ (n+1) i++ ⇒ n	1 (n+1) n
Sum = Sum + i;	n	n
}		
Print ("%d", Sum);	1	1
}		

Time complexity → $3n+4$
$= O(n)$

Space complexity = 3 (constant)

Memory to store 3 variables - Sum, n and i

(2)

	Frequency	Total
Algo Sum 2 {	-	-
Sum = 0	1	1
for (i = 1; i <= n; i++)	i = 1 ⇒ 1 i <= n ⇒ n+1 i++ ⇒ n	1 n+1 n
for (j = 1; j <= n; j++)	j = 1 ⇒ n j <= n ⇒ (n+1)*n j++ ⇒ n*n	n n^2+n n^2
Sum = Sum + j;	n*n	n^2
}	-	-
}	-	-
return Sum;	1	1
}	-	-

Time complexity → $3n^2+4n+4$
$= O(n^2)$

Space complexity ⇒ 4 variables - Sum, i, j, n
$= 4$ (constant)

Page No.
Date

* Correctness of an algorithm :-

2 Methods

1. Using Loop invariants
2. By mathematical inductions

* Loop invariants -

3 things of loop invariants

1. Initialization - It is true prior to the first iteration of the loop.

2. Maintenance - If it is true before an iteration of the loop, it remains true before the next iteration

3. Termination - When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Example ⇒

```
Insertion-Sort (A)
for j = 2 to A.length
    Key = A[j] // insert A[j] into sorted sequence A[1..j-1]
    i = j - 1
    While i > 0 and A[i] > key
        A[i+1] = A[i]
        i = i - 1
    A[i+1] = key
```

Initialization → We start by showing that loop invariant holds before the first loop iteration, when $j=2$. The subarray $A[1..j-1]$ consists of just single element $A[1]$, which is sorted,

Page No.
Date

Correctness of algorithm

- which shows that the loop invariant holds prior to the first iteration of the loop.

Maintenance - Showing that each iteration maintains the loop invariant.
- For loop works by moving A[j-1], A[j-2], A[j-3], --- and so on by one position to the right until it finds proper position for A[j], at which points it inserts the value of A[j].
- The Subarray A[1..j] then consists of the elements originally in A[1..j] but in sorted order.
- Incrementing j for the next iteration of the for loop then preserves the loop invariant.

Termination - Finally we examine what happens when the loop terminates.
- for loop terminates when j > A.length means j > n ie j = n+1. in that case Subarray A[1..n] consists of the elements originally in A[1..n], but in sorted order.
- So we conclude that the entire array is sorted.
- Hence algorithm is correct.

Page No.
Date

Correctness using mathematic induction

3 steps.
1. Basis of induction
2. Inductive Hypothesis
3. Inductive Step.

Example – Sum of the first n positive integer equal to $n(n+1)/2$

Theorm ⇒ $1+2+3+\cdots n = n(n+1)/2$

proof ⇒

① Basis of induction

Since

$S(1) = 1 = 1\cdot(1+1)/2 = 2/2 = 1$

Formula is true for $n=1$

② Inductive Hypothesis

Assume that proof is true for $n=k$

i.e. $S(K) = 1+2+\cdots K = K(K+1)/2$

③ Inductive Step –

Now show that formula is true for $n = K+1$.

- i.e. $S(K+1) = 1+2+\cdots K+K+1 = S(K)+(K+1)$
- Since $S(K) = K(K+1)/2$ By induction Hypothesis, hence

$$S(K+1) = K(K+1)/2 + (K+1)$$

$$= \frac{K}{2}(K+1) + 1\cdot(K+1)$$

$$= \left(\frac{K}{2}+1\right)\cdot(K+1)$$

$$= \left(\frac{K+2}{2}\right)(K+1) = (K+1)(K+2)/2$$

So formula is true for $K+1$.